

Mantar Elektrofizyoloji

Başvuru Ekleri — Deneysel Veriler ve Kaynak Kodlar

Bu belge, mantar (*Pleurotus ostreatus* — istiridye mantarı) üzerinde yürütülen biyoelektrik sinyal ölçüm deneyine ait ek materyalleri kapsamaktadır. Başvuru dosyasının ek bölümünde referans gösterilen tüm görsel ve kaynak kodlar aşağıda sırasıyla sunulmaktadır.

Ek No	İçerik	Sayfa
Ek 1	Deney Düzenegi — Donanım Fotoğrafları	2
Ek 2	Normal Durum Sinyalleri — Zaman Düzlemi	3
Ek 3	Normal Durum Sinyalleri — Frekans Düzlemi	3
Ek 4	Ateş Stresi Sinyalleri — Zaman Düzlemi	4
Ek 5	Ateş Stresi Sinyalleri — Frekans Düzlemi	4
Ek 6	ESP32 Firmware Kodu (Arduino / C++)	5
Ek 7	Python Veri Kayıt ve Analiz Kodu	6

Sistem Mimarisi

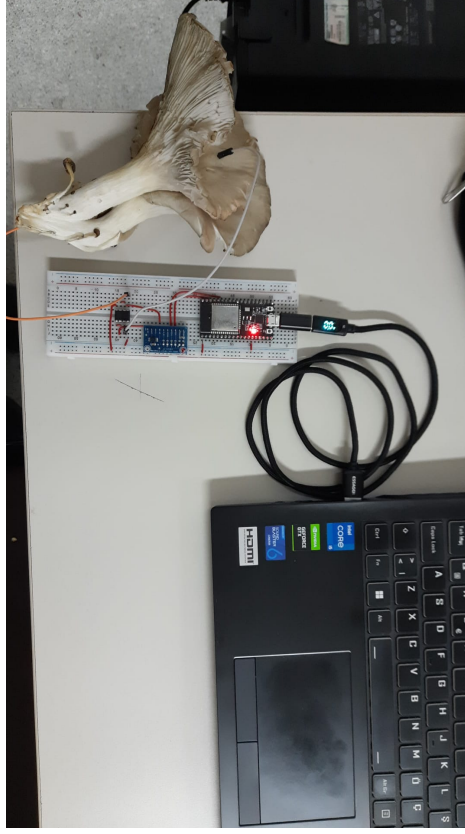
Ölçüm sistemi üç ana bileşenden oluşmaktadır: ESP32 mikrodenetleyici, ADS1115 16-bit diferansiyel ADC modülü ve MCP6022 op-amp tabanlı ön kuvvetlendirici devresi. Mantar dokusuna ince elektrotlar yerleştirilmiş; elektrotlar arası potansiyel fark, GAIN_SIXTEEN modunda (LSB = 7.8125 μ V) ADS1115 üzerinden okunmuştur. Ölçüm verileri UART üzerinden 115200 baud ile bilgisayara aktarılmış, Python tabanlı bir betik ile gerçek zamanlı CSV kaydı yapılmıştır.

Parametre	Değer
Mikrodenetleyici	ESP32 (Xtensa LX6, 240 MHz)
ADC	ADS1115 — 16-bit, diferansiyel giriş
Op-Amp	MCP6022 — düşük gürültü, iki kanallı
PGA Kazancı	GAIN_SIXTEEN — ± 0.256 V tam skala
LSB Çözünürlüğü	7.8125 μ V / bit

Örnekleme Hızı	~45 Hz (delay 20 ms + I2C gecikmesi)
İletişim	UART, 115200 baud, CSV formatında

Ek 1 – Deney Düzeneđi Fotoğrafları

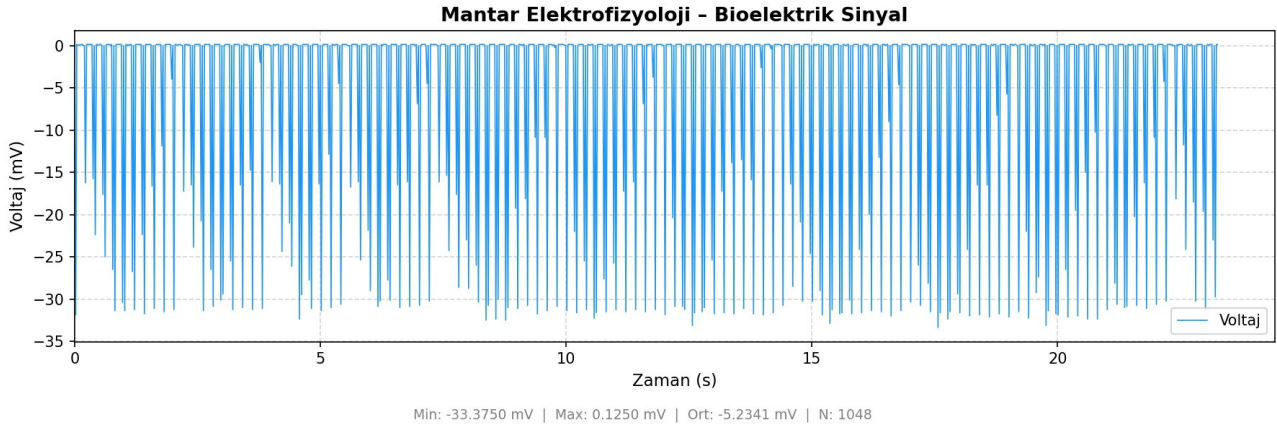
Aşağıdaki fotoğrafta deney düzeneđi görünmektedir. Breadboard üzerinde ESP32, ADS1115 ADC modülü ve MCP6022 op-amp IC'si yer almaktadır. Mantar (*Pleurotus ostreatus*) elektrotlara bađlı halde görölmekte olup tüm sistem dizüstü bilgisayar USB portu üzerinden beslenmektedir.



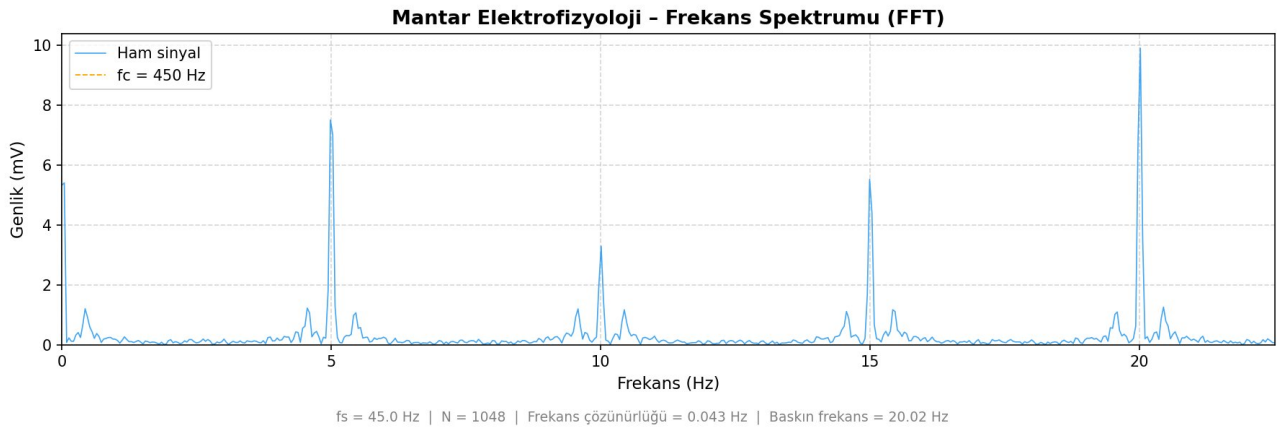
Ek 2 & 3 — Normal Durum Sinyalleri

Aşağıdaki grafikler, mantarın dinlenme halindeki biyoelektrik aktivitesini göstermektedir. Zaman domeninde sinyal -29.4 mV ile $+0.1$ mV aralığında seyrederken ortalama -2.9 mV olarak ölçülmüştür. Periyodik negatif pik yapılarının yaklaşık 5 Hz civarında yoğunlaştığı frekans spektrumundan izlenmektedir.

Ek 2 — Zaman Düzlemi (Normal Durum)



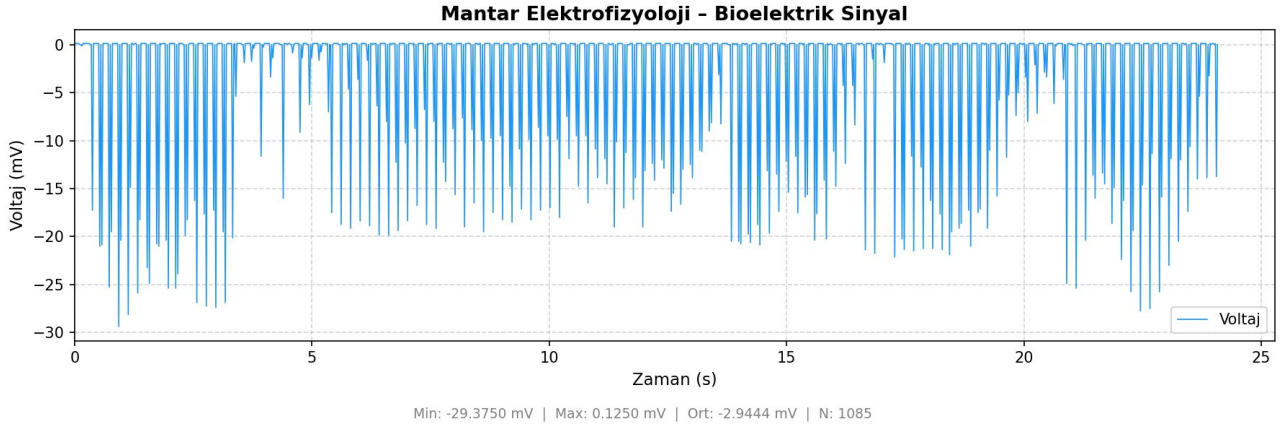
Ek 3 — Frekans Düzlemi (Normal Durum)



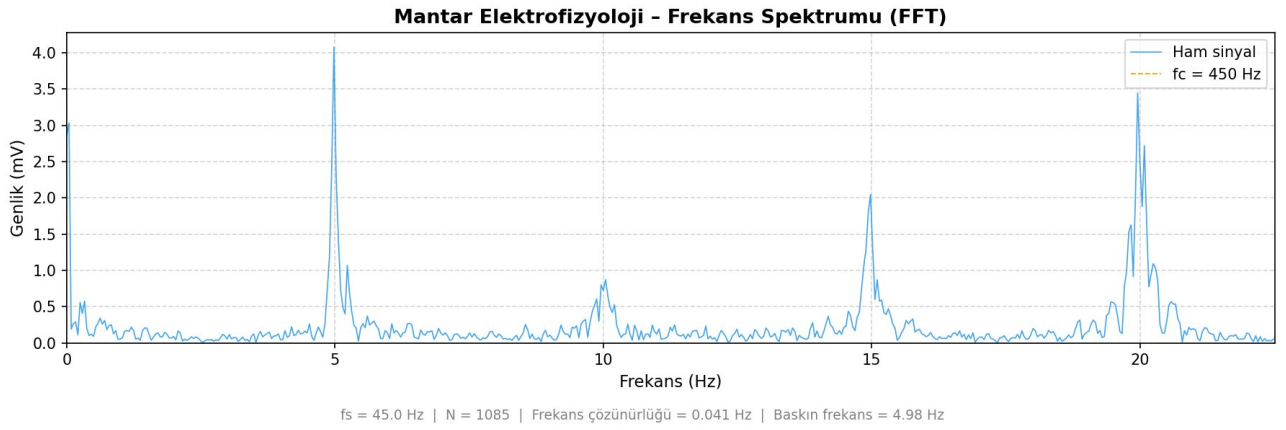
Ek 4 & 5 — Ateş Stresi Sinyalleri

Bu bölümdeki grafikler, mantar dokusuna anlık ateş/ısı stresi uygulanması sırasında kaydedilen biyoelektrik aktiviteyi göstermektedir. Normal duruma kıyasla sinyal genliğinin belirgin biçimde arttığı ve minimum değerin -33.4 mV'a düştüğü görülmektedir. Frekans spektrumunda ise 5 Hz harmoniklerinin genliğinin önemli ölçüde yükseldiği dikkat çekmektedir; bu durum ısı stresinin iyon kanalı aktivitesini artırdığına işaret etmektedir.

Ek 4 — Zaman Düzlemi (Ateş Stresi)



Ek 5 — Frekans Düzlemi (Ateş Stresi)



Ek 6 — ESP32 Firmware Kodu (Arduino / C++)

Aşağıda ESP32 mikrodnetleyicisine yüklenen Arduino IDE uyumlu C++ kaynak kodu yer almaktadır. Kod; ADS1115'i I2C üzerinden başlatmakta, GAIN_SIXTEEN modunda diferansiyel ölçüm yapmakta ve ölçümleri milisaniye zaman damgasıyla birlikte seri porta göndermektedir.

mantar_esp32.ino — Arduino / C++

```
#include <Wire.h>
#include <Adafruit_ADS1X15.h>

Adafruit_ADS1115 ads;

void setup() {
  Serial.begin(115200);
  Wire.begin();
  Wire.setClock(100000);
  if (!ads.begin()) {
    Serial.println("ADS1115 bulunamadi!");
    while (1);
  }
  ads.setGain(GAIN_SIXTEEN);
  Serial.println("Millis,RawValue,Voltage_mV");
}

void loop() {
  int16_t rawValue = ads.readADC_Differential_0_1();
  float voltage_mV = rawValue * 0.0078125;
  Serial.print(millis());
  Serial.print(",");
  Serial.print(rawValue);
  Serial.print(",");
  Serial.println(voltage_mV, 4);
  delay(20);
}
```

Ek 7 — Python Veri Kayıt ve Analiz Kodu

Aşağıda ESP32'den gelen seri veriyi okuyan, CSV dosyasına kaydeden ve kayıt sonrası otomatik olarak zaman / frekans düzlemi grafiklerini oluşturan Python 3 kaynak kodu yer almaktadır. Grafik üretimi için matplotlib, FFT hesabı için NumPy kullanılmıştır.

mantar_elektrofizyoloji.py — Python 3

```
import serial
import csv
import os
import time
from datetime import datetime

# --- AYARLAR ---
PORT = "COM9" # Windows için örn. COM3, Linux için /dev/ttyUSB0
BAUD_RATE = 115200
OUTPUT_FILE = "mantar_elektrofizyoloji.csv"
TIMEOUT = 2 # saniye

def main():
    print(f"[INFO] Seri port açılıyor: {PORT} @ {BAUD_RATE} baud")

    try:
        ser = serial.Serial(PORT, BAUD_RATE, timeout=TIMEOUT)
    except serial.SerialException as e:
        print(f"[HATA] Seri port açılmadı: {e}")
        return

    time.sleep(2) # ESP32 reset sonrası boot süresi

    print(f"[INFO] Kayıt başlıyor → {OUTPUT_FILE}")
    print("[INFO] Durdurmak için Ctrl+C'ye basın.\n")

    try:
        with open(OUTPUT_FILE, "w", newline="", encoding="utf-8") as csvfile:
            writer = csv.writer(csvfile)

            # Başlık satırı
            writer.writerow(["PC_Timestamp", "ESP_Millis", "RawValue", "Voltage_mV"])
            csvfile.flush()

            # İlk satır genellikle başlık veya boş gelir, atla
            ser.readline()

            while True:
                line = ser.readline().decode("utf-8", errors="replace").strip()

                if not line:
                    continue

                # Başlık satırı tekrar gelirse (ESP32 reset) atla
                if line.startswith("Millis"):
                    continue

                parts = line.split(",")
                if len(parts) != 3:
                    print(f"[UYARI] Beklenmeyen format: {line}")
                    continue
```

```
pc_timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")
row = [pc_timestamp] + parts

writer.writerow(row)

# Veri kaybını önle: her satırdan sonra diske yaz
csvfile.flush()
os.fsync(csvfile.fileno())

print(f"{pc_timestamp} | {parts[0]} ms | Raw: {parts[1]:>7} | {parts[2]} mV")

except KeyboardInterrupt:
    print("\n[INFO] Kayıt durduruldu.")
    print(f"[OK] Veriler kaydedildi → {OUTPUT_FILE}")

except serial.SerialException as e:
    print(f"\n[HATA] Seri bağlantı kesildi: {e}")

finally:
    if ser.is_open:
        ser.close()
        print("[INFO] Seri port kapatıldı.")

if __name__ == "__main__":
    main()
```